

BlogApp Projesi

Tasarım ve Etkileşim Geliştirmeleri — Kapsamlı Çalışma Notu (2. Tur)

Admin Şablonu · Girly Tasarım Sistemi · Hover Efektleri · Avatarlar · Okunma Sayısı

Hazırlayan: Serra

19 Ağustos 2026

İçindekiler

1. Giriş: Bu Turda Ne İstendi?
2. Admin Paneli: Ayrı Bir Şablon
3. Tasarım Sistemi: Girly ve Dinamik Estetik
4. Hover ve Geçiş (Transition) Efektleri
5. Yorum Avatarları: Dosya Yüklemeden Kişiselleştirme
6. Okunma Sayısı ve En Çok Okunan Makale
7. Karşılaşılan Bir Sorun: LocalDB Çöküşü ve Kurtarma
8. Web'de Test Süreci
9. Geliştirme Fikirleri
10. Sonuç

BÖLÜM 01

Giriş: Bu Turda Ne İstendi?

Önceki çalışmada BlogApp'e Figma'dan esinlenen siyah-beyaz, tipografi ağırlıklı bir "dergi" teması ("Fikir & Kalem") uygulanmıştı. Bu tur, o temel üzerine sekiz somut istek geldi: admin panelinin görsel olarak ayrı bir şablonda olması, sitenin daha "girly" ve dinamik hissettirmesi, etkileşimli hover efektleri, yorum yazan kişiler için profil görselleri, ana sayfada en çok okunan makalenin öne çıkması, okunma sayısının görünür olması ve son olarak bunların hepsini anlatan bu belge.

Bu belge her özelliği **neden** o şekilde yapıldığını, **hangi dosyalarda** hangi değişikliğin yapıldığını ve **nasil çalıştığını** gerçek kod örnekleriyle anlatır. Ayrıca bu tur sırasında gerçek bir teknik sorunla (yerel veritabanı motorunun çökmesi) karşılaşıldı; o hikaye de Bölüm 7'de ayrıntılıca anlatılıyor çünkü gelecekte benzer bir durumla karşılaşıldığında yol gösterici olacaktır.

BÖLÜM 02

Admin Paneli: Ayrı Bir Şablon

ASP.NET Core MVC'de her `.cshtml` sayfası bir **Layout** (düzen) dosyasının içine "sarılır" — sayfanın kendi içeriği, Layout'taki `@RenderBody()` noktasına yerleştirilir. Projede varsayılan olarak her sayfa `_Layout.cshtml`'i kullanır (bunu `Views/_ViewStart.cshtml` belirler). Admin ekranlarının halka açık dergi görünümünden tamamen ayrışması için ikinci bir layout dosyası, `_AdminLayout.cshtml`, oluşturuldu.

Neden Ayrı Bir Layout, Ayrı Bir CSS Teması Değil?

İki seçenek vardı: (1) aynı Layout'u kullanıp CSS class'larıyla admin sayfalarını farklı göstermek, ya da (2) tamamen ayrı bir Layout dosyası yazmak. İkinci yol seçildi çünkü admin paneli yapısal olarak da farklı: dergi sitesinde üstte yatay bir navbar varken, admin panelinde soldan sabit bir **sidebar (kenar menü)** kullanılıyor. Bu, HTML iskeletinin kendisinin farklı olması gerektiği anlamına gelir — tek bir Layout içinde `if/else` ile yönetmek yerine, iki ayrı dosya çok daha okunabilir ve bakımı kolay bir çözümdür.

Sidebar ve Aktif Sayfa Vurgusu

`_AdminLayout.cshtml` içinde, kullanıcının şu anda hangi sayfada olduğunu tespit edip menüde o linki vurgulayan küçük bir yardımcı fonksiyon tanımlandı:

Views/Shared/_AdminLayout.cshtml (özet)

```
@{
    var currentController = ViewContext.RouteData.Values["controller"]?.ToString();
    var currentAction = ViewContext.RouteData.Values["action"]?.ToString();

    bool IsActive(string controller, string? action = null)
    {
        if (!string.Equals(currentController, controller,
            StringComparison.OrdinalIgnoreCase))
            return false;
        return action == null || string.Equals(currentAction, action,
            StringComparison.OrdinalIgnoreCase);
    }
}

<a asp-controller="Admin" asp-action="Index"
    class="admin-nav-link @(IsActive("Admin", "Index") ? "active" : "")">Panel</a>
```

ViewContext.RouteData.Values — ASP.NET Core'un mevcut isteği hangi Controller/Action'ın karşıladığını tuttuğu yerdir. Bu bilgiyi okuyup her menü linkine **active** class'ı ekleyip eklememeye karar veriyoruz; CSS tarafında **.admin-nav-link.active** altın renkli sol kenarlıkla vurgulanıyor.

Hangi Sayfalar Admin Şablonunu Kullanıyor?

Sekiz görünüm dosyasının her birine tek satırlık bir **Layout = "_AdminLayout"**; eklendi:

- **Views/Admin/Index.cshtml** ve **Views/Admin/Users.cshtml**
- **Views/Category/Index.cshtml**, **Create.cshtml**, **Edit.cshtml**, **Delete.cshtml**
- **Views/Post/Pending.cshtml** (onay bekleyen makaleler)
- **Views/Comment/Index.cshtml** (yorum moderasyonu)

Örnek: *Views/Admin/Index.cshtml* en üstü

```
@{
    ViewData["Title"] = "Yönetim Paneli";
    Layout = "_AdminLayout";
}
```

NOT: **PostController** hem herkese açık (**Index**, **Details**) hem admin'e özel (**Pending**) action'lar içerdiği için, klasör bazlı otomatik bir Layout değişimi (örneğin **Views/Admin/** altına **_ViewStart.cshtml** koymak) yeterli olmazdı — bu yüzden Layout, klasör yerine her görünümde **açıkça** belirtildi. Daha az "sihir", daha kolay takip edilebilir kod.

BÖLÜM 03

Tasarım Sistemi: Girly ve Dinamik Estetik

Siyah-beyaz dergi temasının güçlü tipografi iskeleti korunarak, üzerine bir **vurgu renk paleti** eklendi. Bunun için **site.css** içindeki **:root** bloğuna iki yeni CSS değişkeni tanımlandı:

`wwwroot/css/site.css`

```
:root {
  --ink: #0a0a0a;
  --paper: #ffffff;
  --rose: #c9184a; /* YENİ - girly vurgu rengi */
  --rose-soft: #ffe3ea; /* YENİ - açık pembe ton */
  --gold: #ffd60a; /* YENİ - altın vurgu */
  ...
}
```

CSS değişkeni (custom property) kullanmanın faydası şudur: **--rose** rengini tek bir yerde tanımlayıp site genelinde onlarca yerde **var(--rose)** ile kullanabiliyoruz. Yarın renk paleti değişirse, tek satır değiştirmek yeterli olur — her CSS kuralını tek tek aramaya gerek kalmaz.

Rengın Kullanıldığı Yerler

- Nav linklerinin altına kayan pembe alt çizgi (hover'da)
- Etiket (tag-pill) butonlarının hover rengi — siyahtan pembeye
- Kartların hover'da attığı gölgenin rengi (pembe tonlu, sert siyah değil)
- Ana sayfadaki "En Çok Okunan" rozetinin altın (gold) zemin rengi
- Admin panelinin istatistik kartlarındaki büyük sayılar

Dinamiklik: Giriş Animasyonu

Sayfa yüklendiğinde elemanların aniden belirmesi yerine hafifçe yukarı kayarak belirmesi için basit bir CSS animasyonu eklendi:

wwwroot/css/site.css

```
@keyframes fade-up {
  from { opacity: 0; transform: translateY(14px); }
  to   { opacity: 1; transform: translateY(0); }
}

.fade-up { animation: fade-up 0.6s ease both; }
.fade-up-delay-1 { animation-delay: 0.08s; }
.fade-up-delay-2 { animation-delay: 0.16s; }
```

Ana sayfadaki dev başlık, haber şeridi ve öne çıkan makale bu class'larla art arda (kademeli gecikmeyle) belirir — tek bir animasyon yerine "kademeli" bir giriş hissi yaratır.

Görsel Yer Tutucularda Gradyan

Makale görseli olmadığı için kullanılan siyah kutular, düz renk yerine hafif bir gradyan aldı:

wwwroot/css/site.css — .editorial-media

```
background: linear-gradient(145deg, var(--ink) 0%, #2c0a17 100%);
```

Bu küçük değişiklik, tamamen düz siyah bir yüzeyin "ölü" görünmesini engelleyip hafif bir derinlik hissi katar — girly/dinamik hedefine küçük ama etkili bir katkı.

BÖLÜM 04

Hover ve Geçiş (Transition) Efektleri

Bir hover efekti iki parçadan oluşur: **hedef durum** (`:hover` içindeki stiller) ve **geçiş** (`transition` özelliği — durumlar arası ne kadar sürede ve nasıl bir eğriyle geçileceği). `transition` tanımlanmazsa değişiklik anlık olur, göze sert gelir; tanımlanırsa yumuşak bir canlanma hissi oluşur.

Makale Kartları: Yukarı Kalkma + Gölge

`wwwroot/css/site.css`

```
.editorial-card {  
  transition: transform 0.25s ease, box-shadow 0.25s ease;  
}  
  
.editorial-card:hover {  
  transform: translateY(-6px);  
  box-shadow: 0 18px 30px -18px rgba(201, 24, 74, 0.35);  
}
```

transform: translateY(-6px) kartı 6 piksel yukarı kaydırır; **box-shadow** altına pembe tonlu yumuşak bir gölge düşürür. **transform** ve **box-shadow** özellikleri seçildi çünkü tarayıcı bunları GPU üzerinde işler — **width/margin** gibi "yeniden düzenleme" (reflow) gerektiren özelliklere göre çok daha akıcı çalışır.

Navigasyon Linkleri: Kayan Alt Çizgi

Linklerin altında hover anında soldan sağa "kayan" bir çizgi efekti, gerçek bir çizgi elemanı değil, bir **pseudo-element** (`::after`) ile yapıldı:

`wwwroot/css/site.css`

```
.magazine-nav .nav-link::after {  
  content: "";  
  position: absolute;  
  left: 0.7rem; right: 0.7rem; bottom: 0.15rem;  
  height: 2px;  
  background: var(--rose);  
  transform: scaleX(0);  
  transform-origin: left;  
  transition: transform 0.25s ease;  
}  
  
.magazine-nav .nav-link:hover::after {  
  transform: scaleX(1);  
}
```

Çizgi her zaman DOM'da vardır ama **scaleX(0)** ile genişliği sıfırlanmış, görünmez durumdadır. Hover'da **scaleX(1)** ile soldan başlayarak (**transform-origin: left**) tam genişliğe "açılır". Bu teknik, **width** ile animasyon yapmaktan performans olarak çok daha iyidir.

Etiketler, Butonlar ve Başlıklar

- **Etiket (tag-pill)**: hover'da zemin rengi şeffaftan pembeye dolar, 1px yukarı kalkar.

- **Butonlar (btn-editorial):** hover'da kenarlık ve zemin rengi pembeye döner.
- **Kart başlıkları:** hover'da başlığın altında soldan sağa dolan ince bir çizgi belirir (nav linkiyle aynı `background-size` tekniği, farklı bir uygulamayla).
- **Görsel yer tutucu:** hover'da içindeki büyük baş harf hafifçe büyür ve rengi altına döner (`transform: scale(1.12)`).

NOT: Tüm geçiş süreleri bilinçli olarak 0.15–0.4 saniye aralığında tutuldu. Daha kısa süreler "titrek" hissettirir, daha uzun süreler kullanıcıya yavaş/ağır bir arayüz izlenimi verir.

BÖLÜM 05

Yorum Avatarları: Dosya Yüklemeden Kişiselleştirme

İstenen özellik "yorum yapan kişilerin profil fotoğrafının olması"ydı. Ancak `IdentityUser` modelinde (ve projenin geri kalanında) bir fotoğraf yükleme alt yapısı — dosya yükleme formu, sunucuda saklama, boyut/tip doğrulama, güvenlik kontrolleri — hiç yoktu. Bunu sıfırdan kurmak başlı başına büyük bir özellik olurdu. Bunun yerine, Slack, GitHub ve Gravatar gibi birçok gerçek üründe de görülen bir teknik uygulandı: **e-postadan deterministik olarak üretilen, renkli baş harf avatari**.

AvatarHelper.cs

Helpers/AvatarHelper.cs

```
public static class AvatarHelper
{
    private static readonly string[] Palette =
    {
        "C9184A", "FF4D6D", "7209B7", "B5179E", "F72585", "A4133C", "560BAD"
    };

    public static string GetColor(string? seed)
    {
        if (string.IsNullOrEmpty(seed))
            return Palette[0];

        var hash = 0;
        foreach (var c in seed)
            hash = (hash * 31 + c) & int.MaxValue;

        return Palette[hash % Palette.Length];
    }

    public static string GetInitial(string? seed)
    {
        if (string.IsNullOrEmpty(seed))
            return "?";
        return seed.Trim().Substring(0, 1).ToUpper();
    }
}
```

`GetColor` metodu, e-postadaki her karakterin sayısal (ASCII) değerini 31 ile çarpıp toplar — bu, bilgisayar bilimlerinde çok yaygın kullanılan basit bir **hash (özet) fonksiyonu** deseni (Java'nın

`String.GetHashCode()` metodu da aynı 31 çarpanını kullanır). Elde edilen sayı, renk paletinin uzunluğuna göre `%` (mod) alınarak 0-6 arasına indirgenir. Sonuç: **aynı e-posta her zaman aynı rengi üretir** (deterministik), ama farklı e-postalar farklı renklere dağılır.

Razor Görünümlerinde Kullanımı

Views/Post/Details.cshtml — yorum satırı

```
<span class="avatar"
      style="background: #{@AvatarHelper.GetColor(comment.User?.Email);">
    #{@AvatarHelper.GetInitial(comment.User?.Email)}
</span>
```

Bu yardımcı sınıfın her yerden çağrılabilmesi için `Views/_ViewImports.cshtml` dosyasına `@using BlogApp.Helpers` eklendi — böylece her `.cshtml` dosyasında ayrı ayrı `using` yazmaya gerek kalmadı. Avatar; yorumlarda, makale kartlarında, makale detayında, yazarlar sayfasında ve admin kullanıcı listesinde tutarlı şekilde kullanılıyor.

NOT: Bu yöntemin sınırı: gerçek bir fotoğraf değil, üretilmiş bir simgedir. Gerçek fotoğraf yükleme özelliği eklemek isterseniz Bölüm 9'daki geliştirme fikirlerine bakın.

BÖLÜM 06

Okunma Sayısı ve En Çok Okunan Makale

İki ayrı istek birbiriyle bağlantılı: "okunma sayısının görünmesi" ve "ana sayfada en çok okunan makalenin öne çıkması". İkisi de tek bir yeni veri alanı üzerine kuruldu: `Post.ViewCount`.

Veri Modeli Değişikliği

Models/Post.cs — eklenen satır

```
public int ViewCount { get; set; } = 0;
```

Yeni bir alan eklemek, veritabanı şemasını da değiştirmek anlamına geldiği için bir migration gerekti:

Terminal

```
dotnet ef migrations add AddPostViewCount
dotnet ef database update
```

Migrations/..._AddPostViewCount.cs

```
migrationBuilder.AddColumn<int>(
    name: "ViewCount",
    table: "Posts",
    type: "int",
    nullable: false,
    defaultValue: 0);
```


Sayaç Ne Zaman Artırılır?

PostController.Details action'ında, makale başarıyla görüntülendiğinde sayaç bir artırılır — ama sadece yayındaki (onaylı) makaleler için:

Controllers/PostController.cs — Details action

```
// Yalnızca yayındaki (onaylı) makaleler için gerçek okunma sayısı artırılır.
if (post.Status == PostStatus.Approved)
{
    post.ViewCount++;
    await _context.SaveChangesAsync();
}
```

Bu koşulun nedeni şu: bir yazarın kendi onay bekleyen taslağını kontrol etmesi ya da admin'in onay öncesi bir makaleyi incelemesi, gerçek bir "okuyucu okuması" sayılmamalı. Sayaç sadece herkese açık, gerçekten yayınlanmış içerik için anlamlıdır.

Ana Sayfada "En Çok Okunan" Seçimi

HomeController.Index, artık tek bir liste yerine iki ayrı sorgu çalıştırıyor: en yüksek **ViewCount**'a sahip onaylı makaleyi (öne çıkan) ve geri kalan en yeni makaleleri (diğer makaleler ızgarası):

Controllers/HomeController.cs (özet)

```
var featured = await _context.Posts
    .Include(p => p.User)
    .Include(p => p.Category)
    .Where(p => p.Status == PostStatus.Approved)
    .OrderByDescending(p => p.ViewCount)
    .ThenByDescending(p => p.CreatedDate)
    .FirstOrDefaultAsync();

var latestPosts = await _context.Posts
    .Where(p => p.Status == PostStatus.Approved && p.Id != featured.Id)
    .OrderByDescending(p => p.CreatedDate)
    .Take(6)
    .ToListAsync();
```

ThenByDescending(p => p.CreatedDate) bir **eşitlik bozucu (tie-breaker)** kuralıdır: eğer iki makale de sıfır (veya eşit) okunmaya sahipse, aralarından en yeni olan öne çıkar. Bu sayede sitenin ilk açıldığı gün (henüz hiç okunma yokken) bile mantıklı bir makale öne çıkarılmış olur.

Neden Yeni Bir ViewModel Gerekti?

Önceki tasarımda **Home/Index.cshtml**, listedeki ilk makaleyi "öne çıkan" olarak kabul ediyordu (**Model.FirstOrDefault()**). Artık öne çıkan makale **en çok okunana** göre seçildiğinden, bu iki bilgiyi (öne çıkan + diğerleri) ayrı ayrı taşıyan yapılandırılmış bir ViewModel gerekti:

Models/ViewModels/HomeViewModel.cs

```
public class HomeViewModel
{
    public Post? FeaturedPost { get; set; }
    public List<Post> LatestPosts { get; set; } = new List<Post>();
}
```

Okunma Sayısının Gösterimi: Tekrar Kullanılabilir Partial View

Göz simgesi + sayı kombinasyonu; makale kartlarında, makale detayında, öne çıkan makalede ve "Makalelerim" tablosunda — dört farklı yerde tekrarlanıyordu. Bu kod tekrarını önlemek için küçük bir Partial View'a taşındı (Bölüm 8'deki `_PostForm` ile aynı DRY mantığı):

Views/Shared/_ViewCount.cshtml

```
@model int
<span class="view-count">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
        <path d="M1 12s4-8 11-8 11 8 4 8-11 8-11-8z"></path>
        <circle cx="12" cy="12" r="3"></circle>
    </svg>
    @Model
</span>
```

Kullanım örneği

```
<partial name="_ViewCount" model="@post.ViewCount" />
```

BÖLÜM 07

Karşılaşılan Bir Sorun: LocalDB Çöküşü ve Kurtarma

`ViewCount` migration'ı veritabanına uygulanmaya çalışılırken, projenin kodundan tamamen bağımsız, ortamsal bir sorunla karşılaşıldı. Bu bölüm, sorunun nasıl teşhis edilip veri kaybı olmadan çözüldüğünü adım adım anlatıyor — ileride benzer bir LocalDB sorunuyla karşılaşıldığında izlenecek yol haritası olarak düşünülebilir.

Belirti

Terminal çıktısı (özetlenmiş)

```
dotnet ef database update
# Hata: A network-related or instance-specific error occurred...
# Local Database Runtime error occurred. SQL Server process failed to start.
```

İlk Denemeler Yetersiz Kaldı

Terminal — instance yeniden başlatma/oluşturma denemeleri başarısız

```
sqllocaldb stop mssqllocaldb
sqllocaldb start mssqllocaldb
# Start of LocalDB instance failed...

sqllocaldb delete mssqllocaldb
sqllocaldb create mssqllocaldb
# Cannot create an automatic instance...
```

Basit "kapat-aç" ve "sil-yeniden oluştur" adımları sorunu çözmedi. Bu noktada, hatanın gerçek kök nedenini görmek için Windows'un **Application Event Log**'una (uygulama olay günlüğü) bakıldı.

Kök Neden: Kilitli Bir Dosya

PowerShell — olay günlüğü incelemesi

```
Get-WinEvent -LogName Application -MaxEvents 30 |
  Where-Object { $_.Message -like "*LocalDB*" }

# Windows API call CopyFileW returned error code: 32.
# Dosya başka bir işlem tarafından kullanıldığından bu işlem dosyaya erişemiyor.
```

"Error code: 32" Windows'ta klasik bir **dosya kilidi** hatasıdır: başka bir süreç o dosyayı hâlâ açık tutuyordur. Kim olduğunu bulmak için çalışan SQL Server süreçleri listelendi:

PowerShell

```
Get-Process -Name sqlservr

#      Id ProcessName StartTime
#      --
# 22684 sqlservr      19.08.2026 10:48:29
```

Gerçekten de, önceki bir başlatma denemesinden kalma **unutulmuş (zombie) bir sqlservr.exe süreci** hâlâ çalışıyor ve veritabanı dosyalarını kilitli tutuyordu — muhtemelen daha önceki bir oturumda **dotnet run** sürecinin **Stop-Process -Force** ile zorla sonlandırılması, alt sürecini (SQL Server'ı) yetim bırakmıştı.

PowerShell — sorunun çözümü

```
Stop-Process -Id 22684 -Force
sqllocaldb start mssqllocaldb
# LocalDB instance "mssqllocaldb" started. □
```

İkinci Sorun: Silinen Instance, Kaybolmayan Veri

Instance başarıyla başladı, ama **dotnet ef database update** bu kez farklı bir hatayla karşılaştı:

Hata mesajı

```
Cannot create file 'C:\Users\SERRA\BlogAppDb.mdf' because it already exists.
```

Bunun nedeni şuydu: **sqllocaldb delete** komutu instance'ın **katalogunu** (hangi veritabanlarının bağlı olduğu kaydını) sildi, ama fiziksel **.mdf/.ldf** dosyaları diskte olduğu gibi kaldı. EF Core,

veritabanının "var olmadığını" sanıp sıfırdan oluşturmaya çalıştı — ama aynı isimde bir dosya zaten mevcut olduğu için başarısız oldu.

Çözüm: var olan veri dosyalarını yeni instance'a **elle yeniden bağlamak (attach)**:

Terminal — veritabanını yeniden bağlama

```
sqlcmd -S "(localdb)\mssqllocaldb" -Q "  
CREATE DATABASE BlogAppDb  
ON (FILENAME = N'C:\Users\SERRA\BlogAppDb.mdf'),  
    (FILENAME = N'C:\Users\SERRA\BlogAppDb_log.ldf')  
FOR ATTACH;"
```

Bu komuttan sonra tüm eski veriler (4 makale, yorumlar, kullanıcı hesapları) ve migration geçmişisi bir sorgu ile doğrulandı — **hiçbir veri kaybı olmadı**. Ardından bekleyen **AddPostViewCount** migration'ı normal şekilde uygulandı.

NOT: Bu olayın en önemli çıkarımı: LocalDB'de **"instance" (motor süreci) ile "veritabanı dosyaları" (.mdf/.ldf) birbirinden bağımsızdır**. Bir instance silinse bile fiziksel veri dosyaları diskte kalır ve **FOR ATTACH** ile her zaman geri kazanılabilir — bu yüzden LocalDB sorunlarında panikleyip veritabanını sıfırdan kurmadan önce, dosyaların hâlâ diskte olup olmadığını kontrol etmek her zaman ilk adım olmalı.

BÖLÜM 08

Web'de Test Süreci

Kod derlendikten (**dotnet build** → 0 hata, 0 uyarı) ve migration veritabanına uygulandıktan sonra, her özellik tarayıcıda tek tek doğrulandı.

- Ana sayfa: girly renk paleti, fade-up giriş animasyonu ve "☐ En Çok Okunan" rozeti doğru makalede görüldü.
- Bir makale iki kez ziyaret edildi; okunma sayısının **0 → 1 → 2** şeklinde arttığı doğrulandı (konsol hatası yok).
- Admin olarak giriş yapıp **/Admin** adresine gidildiğinde, sekme başlığının "BlogApp Admin" olduğu ve sol kenar menüsünün (Panel, Kullanıcılar, Kategoriler, Onay Bekleyenler, Yorumlar) doğru görüldüğü kontrol edildi.
- Admin'in sekiz farklı sayfası (Panel, Kullanıcılar, Kategoriler, Kategori Ekle, Onay Bekleyenler, Yorumlar) tek tek gezilip hepsinin yeni sidebar şablonunu kullandığı, hiçbir konsol hatası çıkmadığı doğrulandı.
- Avatarların yorumlarda, makale kartlarında, yazar sayfasında ve admin kullanıcı listesinde tutarlı ve doğru renklerle görüldüğü kontrol edildi (JavaScript ile hesaplanmış CSS değerleri okunarak: **border-radius: 50%**, arka plan rengi paletten birebir eşleşiyor).
- Hem halka açık site hem admin paneli, mobil genişlikte (375px) test edildi — hiçbir sayfada yatay taşma (**horizontalOverflow: false**) tespit edilmedi; admin kenar menüsünün mobilde tam genişliğe yayılarak üstte konumlandığı doğrulandı.

NOT: Görsel render (ekran görüntüsü) bu ortamda alınamadığından, doğrulamalar `getComputedStyle` ile gerçek CSS değerlerini (renk, geçiş süresi, genişlik, taşma durumu) okuyarak yapıldı. Bu, "kod çalışıyor görünüyor"dan daha güçlü bir kanıt sağlar çünkü tarayıcının gerçekten ne hesapladığını doğrudan gösterir.

BÖLÜM 09

Geliştirme Fikirleri

Bu bölüm, projeyi bir sonraki seviyeye taşımak isteyen biri için somut, uygulanabilir öneriler sunar.

1. Gerçek Profil Fotoğrafı Yükleme

Şu anki baş harf avaturları yerine gerçek fotoğraf yüklemek için: formlara `IFormFile` tipinde bir alan eklenir, dosya `wwwroot/uploads/avatars/` klasörüne (veya üretim ortamında bir bulut depolama servisine, ör. Azure Blob Storage) kaydedilir, dosya boyutu ve uzantısı (`.jpg`, `.png`) sunucu tarafında doğrulanır, `IdentityUser`'a ek bir `AvatarUrl` alanı eklenmesi için `Identity`'nin `ApplicationUser : IdentityUser` şeklinde özelleştirilmesi gerekir.

2. Okunma Sayısını Tekilleştirme

Şu anki sayaç, aynı kişi sayfayı her yenilediğinde artıyor. Gerçekçi bir "benzersiz okuyucu" sayısı için, kullanıcının o makaleyi zaten görüntülediğini bir çerez (cookie) veya oturum (session) anahtarıyla hatırlayıp, aynı ziyaretçi için tekrar saymamak gerekir.

3. Zaman Aralıklı "En Çok Okunanlar"

Şu an "en çok okunan", makalenin **tüm zamanlar** toplam sayısına göre seçiliyor. Ayrı bir `PostView` tablosu (her görüntülemeyi tarihiyle birlikte kaydeden) eklenirse, "bu hafta en çok okunanlar" veya "bu ay en çok okunanlar" gibi zaman bazlı listeler `GROUP BY` ile hesaplanabilir.

4. Admin Panelinde Grafikler

Chart.js gibi hafif bir JavaScript kütüphanesiyle, admin panelindeki sayısal istatistik kartlarının yanına zaman içindeki kayıt/makale/yorum trendlerini gösteren basit çizgi veya çubuk grafikler eklenebilir.

5. Arama ve Sayfalama

Makale sayısı arttıkça, `Post/Index` sayfası artık tüm makaleleri tek seferde listelemek yerine `Skip()/Take()` ile sayfanmalı; başlık/içerik üzerinde basit bir arama kutusu eklenmesi de kullanılabilirliği ciddi şekilde artırır.

6. Bildirimler

Bir yazarın makalesi onaylandığında veya reddedildiğinde kendisine haber verilmesi (e-posta ile ya da uygulama içi bir bildirim simgesiyle), onay sürecini daha şeffaf hale getirir.

7. Otomatik Testler

Bu projedeki tüm doğrulamalar tarayıcıda elle yapıldı. Bir sonraki adım, `PostController`, `CommentController` gibi sınıflar için **unit testler** (xUnit + Moq ile) ve uçtan uca senaryolar için **entegrasyon testleri** (`WebApplicationFactory`) eklemek olabilir — bu, gelecekteki değişikliklerin mevcut özellikleri bozmadığından emin olmayı otomatikleştirir.

8. Karanlık Mod (Dark Mode)

Tüm renkler zaten CSS değişkenleri (`:root` içinde) olarak tanımlandığı için, `prefers-color-scheme: dark` medya sorgusuyla alternatif bir renk seti tanımlamak nispeten az efor gerektirir — tasarım sistemi buna hazır bir temel sağlıyor.

BÖLÜM 10

Sonuç

İstek	Durum
Admin panelinin ayrı bir şablonda olması	Tamamlandı — <code>_AdminLayout.cshtml</code> , 8 görünümde uygulandı
Daha dinamik ve girly estetik	Tamamlandı — rose/gold vurgu renkleri, fade-up animasyonu, gradyanlar
Hover efektleri	Tamamlandı — kart kalkması, link altı çizgisi, etiket/buton renk geçişleri
Yorum yapanların profil görseli	Tamamlandı — deterministik baş harf avatarı (AvatarHelper)
Ana sayfada en çok okunan makale öne çıksın	Tamamlandı — ViewCount'a göre sıralanan öne çıkan makale
Okunma sayısının görünmesi	Tamamlandı — kart, detay ve Makalelerim sayfalarında
Kapsamlı açıklayıcı PDF	Tamamlandı — bu belge

Tüm değişiklikler sonrasında proje `dotnet build` ile **0 hata, 0 uyarı** vermeye devam ediyor. Süreç içinde karşılaşılan LocalDB çökmesi, projenin kodundan bağımsız bir ortam sorunuymuş ve veri kaybı olmadan çözüldü. Tüm yeni özellikler tarayıcıda uçtan uca test edildi ve beklendiği gibi çalıştığı doğrulandı.